

TEACH YOURSELF LECTURE

Lecture 03

Programming Fundamentals

Chapter lecture — 24 topics with explanations, worked examples, Urdu notes, practice questions and a full answer key.

Computer Science XI · Bilingual support: English + اردو

کمپیوٹر سائنس — گیارہویں جماعت: خود سیکھیں، خود پرکھیں

24 teach-yourself lectures in this pack

★ Golden topics, Urdu notes, review and answer key

XI

How to use this lecture

Learn this chapter **on your own**. Read *Learn it*, try the worked example on paper, then answer *Check yourself* before looking at the answers. The green box repeats the idea in Urdu. ★ Golden topics are high-yield in board exams.

یہ لیکچر خود سیکھیں۔ مثال کاغذ پر حل کریں، پھر جواب دیکھنے سے پہلے خود کو پرکھیں۔

Lectures in this pack

Chapter 3: Programming Fundamentals. Study one lecture at a time, then use the chapter review and answer key at the end of this PDF.

L01. 3.1 Programming

L03. 3.2 Career Growth with Python

L05. 3.3.1 Visual Studio Code (VS Code)

L07. 3.4.1 Variables

L09. 3.4.2 Input and Output Handling

L11. 3.4.3 Relational, Logical and Membership Operators

L13. 3.5 Control Structures

L15. 3.5.2 Selection Statements: if and if-else

L17. 3.5.3 Repetition Statements: the for Loop and range()

L19. 3.5.3 Nested Loops and for vs while

L21. 3.6 Libraries in Python

L23. 3.6.2 Third-Party Libraries

L02. 3.1.1 Programming Language Types

L04. 3.3 Integrated Development Environment (IDE)

L06. 3.4 Fundamentals of Python Programming

L08. 3.4.1 Data Types in Python

L10. 3.4.3 Operators and Operands: Arithmetic and Assignment

L12. 3.4.3 Bitwise Operators

L14. 3.5.1 Sequence

L16. 3.5.2 Selection Statements: if-elif-else and Nested if

L18. 3.5.3 The while Loop, break and continue

L20. Practical Activity: Shopping Bill Calculator

L22. 3.6.1 Built-in Libraries

L24. 3.7 Debugging Code in Python

CHAPTER 3

Programming Fundamentals

In this chapter you start writing real programs in Python. You will learn how programs store data, take input, make decisions, repeat work, use ready-made libraries, and how to find and fix your own mistakes.

اس باب میں آپ پائتھن (Python) میں حقیقی پروگرام لکھنا سیکھیں گے۔ آپ جانیں گے کہ پروگرام ڈیٹا کیسے محفوظ کرتا ہے، فیصلے کیسے کرتا ہے، کام کو کیسے دہراتا ہے، اور غلطیاں کیسے تلاش کی جاتی ہیں۔

What you will learn

- Define a program and a programming language, and compare low-, mid- and high-level languages.
- Describe the features of an IDE and the main parts of the VS Code window.
- Take input with `input()`, convert it with `int()` / `float()`, and display output with `print()` and f-strings.
- Use sequence, selection (`if`, `if-else`, `if-elif-else`, nested `if`) and repetition (`for`, `while`, `range()`, `break`, `continue`, nested loops).
- Identify syntax, runtime and logical errors and read a traceback to fix a bug.
- Name career areas where Python is used.
- Write Python programs that use comments, correct indentation, valid variable names and the main data types.
- Evaluate expressions with arithmetic, assignment, relational, logical, membership and bitwise operators, including binary working.
- Import and use built-in libraries (`math`, `random`, `datetime`) and install third-party libraries with `pip`.

3.1 Programming

LEARN IT

Computers are "dumb" machines. They are very fast, but they cannot think or do anything on their own. They need exact, step-by-step **instructions**.

- A set of instructions given to a computer is called a **program**.
- The language used to write these instructions is called a **programming language**.
- Writing programs is called **programming**, and the person who writes them is a **programmer**.

Think of a recipe. A cook follows the recipe line by line: "boil water, add rice, wait 15 minutes". A computer follows a program in the same way, one instruction at a time, without guessing what you meant.

WORKED EXAMPLE

A tiny program. Here is a complete Python program with one instruction. It tells the computer to show a message on the screen.

```
print("Hello")
```

OUTPUT

```
Hello
```

The word `print` is the instruction; `"Hello"` is the data it works on.

REMEMBER IT

Program = recipe, programmer = recipe writer, computer = cook who follows every line exactly. If the recipe is wrong, the food is wrong. The cook does not fix it for you.

COMMON MISTAKE

Students often think the computer "understands" what they want. It does not. If you type `Print("Hello")` with a capital P, Python stops with an error, because it only knows `print`. Be exact.

کمپیوٹر خود سے کچھ نہیں سوچ سکتا۔ اسے کام کرنے کے لیے ہدایات چاہئیں۔ ہدایات کے مجموعے کو پروگرام (Program) کہتے ہیں، اور جس زبان میں یہ ہدایات لکھی جاتی ہیں اسے پروگرامنگ لینگویج (Programming Language) کہتے ہیں۔

CHECK YOURSELF

1. What is a program?
2. Why do we say computers are "dumb" machines?

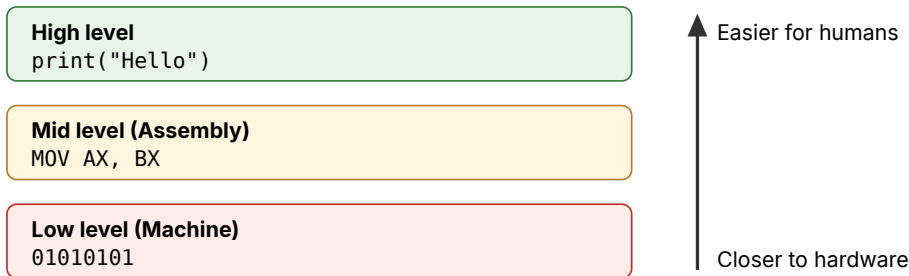
Answers: 1. A set of instructions given to a computer to perform a task. 2. Because they cannot think or do anything without instructions; they only follow exactly what they are told.

3.1.1 Programming Language Types

LEARN IT

Programming languages are classified into **three main types**, depending on how close they are to the computer's own language (binary 0s and 1s) or to human language.

Type	Description	Examples
Low level	Closest to the native language of the computer (binary 0s and 1s). Very difficult for humans to read and write.	Machine language (binary)
Mid level	Bridges the gap. It can access hardware but is easier to read than binary because it uses short words called mnemonics (e.g. <code>MOV</code> , <code>ADD</code>).	Assembly language
High level	Designed to be easy for humans to read and write. Uses English-like words.	C++, Java, Python



The three levels of programming languages, with a sample line from each.

Common high-level languages

Language	Key idea	Used for
C++	Powerful and closer to hardware.	System software, operating systems, high-performance computer games.
Java	"Write once, run anywhere."	Large enterprise applications, Android mobile apps, server programming.
Python	Simple, readable syntax.	Data Science, Artificial Intelligence, Web Development, and learning to program.

WORKED EXAMPLE

Same task, three levels. Suppose you want to copy a value.

- Machine language: a pattern like 01010101 — you would need a chart to know what it means.
- Assembly: MOV AX, BX — "move the value in register BX into register AX". Shorter words, but you still need to know the hardware.
- Python: a = b — you simply say "a gets the value of b".

The higher the level, the less you need to know about the hardware and the faster you can write the program.

REMEMBER IT

"Low = Like the machine, High = Human." Mid level sits in the Middle and uses Mnemonics (both start with M).

COMMON MISTAKE

"Low level" does not mean "bad" or "weak". Low-level code runs very fast and controls hardware directly. It is just harder for humans to write.

اردو میں سمجھیں

پروگرامنگ زبانوں کی تین اقسام ہیں: لو لیول (Low Level) جو بائری 0 اور 1 میں ہوتی ہے، مڈ لیول (Mid Level) یعنی اسمبلی جو مختصر الفاظ (Mnemonics) استعمال کرتی ہے، اور ہائی لیول (High Level) جیسے پائتھن، جاوا اور سی پلس پلس جو انگریزی جیسے الفاظ استعمال کرتی ہیں اور انسانوں کے لیے آسان ہیں۔

CHECK YOURSELF

1. Which level is MOV AX, BX?
2. Which high-level language is famous for "write once, run anywhere"?
3. Give one use of C++.

Answers: 1. Mid level (assembly language). 2. Java. 3. Any one: system software, operating systems, high-performance games.

3.2 Career Growth with Python

LEARN IT

Python is one of the most in-demand languages in the world. Learning it opens many career paths:

- **Software Development:** build desktop and web applications.
- **Data Science and Data Analysis:** analyse data, create reports and make predictions.
- **Artificial Intelligence and Machine Learning:** develop intelligent systems, chatbots and recommendation algorithms (like "videos you may like").
- **Web Development:** use frameworks such as **Django** and **Flask** to build websites.
- **Automation and Scripting:** automate repetitive office or lab tasks (renaming 500 files, filling spreadsheets).
- **Cybersecurity and Networking:** write security tools and analyse network data.

Why Python? Python lets you focus on **problem-solving and logic** instead of memorising complex syntax rules. Compare printing a message in Python with C++:

```
# Python
print("Hello")
```

```
// C++
#include <iostream>
int main() {
    std::cout << "Hello";
    return 0;
}
```

WORKED EXAMPLE

Automation example. A school clerk must write "Roll No. 1" to "Roll No. 5" on a list. Python does it in two lines:

```
for n in range(1, 6):
    print("Roll No.", n)
```

OUTPUT

```
Roll No. 1
Roll No. 2
Roll No. 3
Roll No. 4
Roll No. 5
```

Change 6 to 501 and it writes 500 lines just as fast. (You will learn loops in 3.5.3.)

REMEMBER IT

Six careers — remember **"S-D-A-W-A-C"**: **S**oftware, **D**ata, **A**I, **W**eb, **A**utomation, **C**ybersecurity.

COMMON MISTAKE

Django and Flask are **web frameworks** (for websites), not AI tools. AI/ML uses libraries such as TensorFlow and PyTorch (see 3.6.2).

اردو میں سمجھیں

پائتھن سیکھ کر آپ سافٹ ویئر ڈیولپمنٹ، ڈیٹا سائنس، مصنوعی ذہانت (AI)، ویب ڈیولپمنٹ، آٹومیشن اور سائبر سیکیورٹی جیسے شعبوں میں کیریئر بنا سکتے ہیں۔ پائتھن کا آسان انداز آپ کو مشکل اصول یاد کرنے کے بجائے مسئلہ حل کرنے پر توجہ دینے دیتا ہے۔

CHECK YOURSELF

1. Name two Python web frameworks.
2. Which career area builds chatbots and recommendation systems?

Answers: 1. Django and Flask. 2. Artificial Intelligence and Machine Learning.

3.3 Integrated Development Environment (IDE)

LEARN IT

An **IDE (Integrated Development Environment)** is a software tool that helps programmers **write, run and manage code in one place**. It is a complete workspace for programming. "Integrated" means many tools are joined together in one window.

Key features of an IDE:

- **Code Editor:** where you write and edit code.
- **Syntax Highlighting:** colours different parts of the code (keywords, strings, numbers) to make it easier to read.
- **Auto-completion:** suggests code while you type (type `pri` and it offers `print`).
- **Run/Execute Tool:** runs the program with one click.
- **Debugger:** helps you find and fix errors, for example by pausing the program at a chosen line.

WORKED EXAMPLE

Without vs with an IDE. Without an IDE you would write code in Notepad, save it, open a terminal and type `python file.py`, then go back to Notepad to fix mistakes. With an IDE you write, run (press the ► Run button) and debug in the same window, and errors are underlined while you type.

REMEMBER IT

IDE features — "**E-S-A-R-D**": Editor, Syntax highlighting, Auto-completion, Run, Debugger.

COMMON MISTAKE

An IDE does not write the program for you and does not catch logical mistakes. It only makes writing, running and debugging easier.

اردو میں سمجھیں

آئی ڈی ای (IDE) ایک ایسا سافٹ ویئر ہے جس میں کوڈ لکھنا، چلانا اور غلطیاں درست کرنا ایک ہی جگہ ممکن ہوتا ہے۔ اس کی اہم خصوصیات میں کوڈ ایڈیٹر، رنگین کوڈ (Syntax Highlighting)، خودکار تجاویز (Auto completion)، رن بٹن اور ڈیباگر شامل ہیں۔

CHECK YOURSELF

1. What does "integrated" mean in IDE?
2. Which IDE feature suggests code while you type?

Answers: 1. Many programming tools (editor, runner, debugger) are combined in one program. 2. Auto-completion.

3.3.1 Visual Studio Code (VS Code)

LEARN IT

VS Code is a **free, lightweight and powerful code editor** made by Microsoft and widely used for Python. By default it is **not a full IDE**, but it becomes one when you install **extensions** (add-ons), such as the **Python extension by Microsoft**.

VS Code interface elements

Element	Description
1. Top Menu Bar	Control over files, editing, navigation and workspace (File, Edit, Selection, View, Go, Run).
2. Side Bar	Contains the core tools: Explorer, Search, Source Control, Run & Debug and Extensions.
3. Explorer	Shows project files and folders. Helps you create, open and manage your Python files.
4. Search	Finds and replaces text in the open file or across the whole project.
5. Source Control	Connects with Git (a version control system) to track changes, undo mistakes and collaborate.
6. Run & Debug	Runs code, finds errors, pauses at specific lines and lets you inspect variables.
7. Extensions	Like an App Store for VS Code. Add features such as the Python extension, Prettier or Python Debugger.



A simplified VS Code window: menu bar, side bar, Explorer and the code editor.

WORKED EXAMPLE

Setting up and running your first program in VS Code:

Install Python → Install VS Code → Extensions: install "Python" (Microsoft) →
Explorer: new file `hello.py` → Type code → Click ► Run

```
print("Hello, World!")
```

OUTPUT
Hello, World!

The output appears in the **Terminal** panel at the bottom of the window.

REMEMBER IT

Extensions are the **"App Store"** of VS Code: just as apps give your phone new abilities, extensions give VS Code new abilities.

COMMON MISTAKE

Always save Python files with the `.py` extension (e.g. `hello.py`). Without it, VS Code does not know the file is Python, so there is no syntax highlighting and no Run button.

اردو میں سمجھیں

وی ایس کوڈ (VS Code) ایک مفت اور ہلکا پھلکا کوڈ ایڈیٹر ہے۔ یہ اپنے آپ میں مکمل آئی ڈی ای نہیں، لیکن ایکسٹینشنز (Extensions) لگانے سے آئی ڈی ای بن جاتا ہے۔ اس کے اہم حصے مینو بار، سائیڈ بار، ایکسپلورر، سرچ، سورس کنٹرول، رن اینڈ ڈیبگ اور ایکسٹینشنز ہیں۔

CHECK YOURSELF

1. Which VS Code element connects to Git?
2. Is VS Code a full IDE by default? How does it become one?
3. Where do you see and manage your project's files?

Answers: 1. Source Control. 2. No; it becomes an IDE by installing extensions (e.g. the Python extension). 3. Explorer.

3.4 Fundamentals of Python Programming ★ Golden

LEARN IT

The basic structure of a Python program is simple and clear. A program takes data, **processes it using statements**, and finally **shows output**. Python runs the statements **top to bottom, one after another** (sequentially).

A **statement** is one complete instruction, usually written on one line, such as `print("Hi")` or `x = 5`.

1. **Comments (#)** — Comments are notes written in the code **for humans**. They start with the `#` symbol. The Python interpreter completely **ignores** them. Use them to explain what your code does.
2. **Indentation (spaces)** — C++ and Java use curly brackets `{ }` to mark a **block** of code (a group of statements that belong together). Python instead uses **indentation**: spaces at the beginning of a line. Indentation is **mandatory** in Python. The standard is **4 spaces** (or 1 tab).

```
# This is a comment. It explains the code below.
print("Hello, World!")
if 5 > 2:
    # The space before 'print' is the indentation
    print("Five is greater than two!")
```

OUTPUT

```
Hello, World!
Five is greater than two!
```

WORKED EXAMPLE

Reading the program line by line:

1. Line 1 starts with `#`, so Python skips it.
2. Line 2 prints `Hello, World!`.
3. Line 3 asks a question: is 5 greater than 2? The colon `:` says "a block follows".
4. Line 4 is a comment inside the block (ignored).
5. Line 5 is indented by 4 spaces, so it belongs to the `if` block. Because `5 > 2` is true, it runs.

What happens if you forget the indentation?

```
if 5 > 2:
print("Five is greater than two!")
```

OUTPUT

```
File "indent.py", line 2
  print("Five is greater than two!")
  ^
IndentationError: expected an indented block after 'if' statement on line 1
```

Python refuses to run the program and points to line 2. Add 4 spaces before `print` to fix it.

REMEMBER IT

Think of indentation like the points and sub-points in your notebook. Sub-points are written a little to the right to show they "belong" to the main point. A colon `:` at the end of a line always means "**indent the next line**".

COMMON MISTAKE

IndentationError? Check your spaces. **Mixing tabs and spaces** can break a Python program even when the code looks lined up. Choose 4 spaces and use them everywhere (VS Code inserts 4 spaces when you press Tab in a `.py` file).

اردو میں سمجھیں

پائتھن پروگرام اوپر سے نیچے ترتیب سے چلتا ہے۔ `#` سے شروع ہونے والی لائن تبصرہ (Comment) ہوتی ہے جسے پائتھن نظرانداز کر دیتا ہے۔ پائتھن میں کوڈ کے بلاک کو بریکٹ کے بجائے انڈینٹیشن (Indentation) یعنی لائن کے شروع میں چار خالی جگہوں سے ظاہر کیا جاتا ہے، اور یہ لازمی ہے۔

EXAM TIP

Common questions: "What is the purpose of comments?", "Why is indentation important in Python?", "How is Python different from C++/Java in defining blocks?" Write: comments start with `#` and are ignored by the interpreter; Python uses indentation (4 spaces) instead of `{ }`; wrong indentation gives an `IndentationError`. Always give a 3–4 line code example.

CHECK YOURSELF

1. Predict the output:

```
# print("A")
print("B")
if 3 > 7:
    print("C")
print("D")
```

2. What is the standard amount of indentation in Python?

Answers: 1. B then D on separate lines (line 1 is a comment; `3 > 7` is false so C is skipped). 2. 4 spaces (or 1 tab).

3.4.1 Variables ★ Golden

LEARN IT

A **variable** is a **named space in computer memory used to store data**. Think of it as a labelled container (a box with a name written on it). You put a value in it with the **assignment operator** `=`, which means "store", not "equals".

In Python you **do not need to declare the type** of a variable. Python figures it out automatically from the value. This is called **dynamic typing**: the same variable can later hold a different type.

Rules for naming variables:

- Can contain **letters, numbers and underscores** (`_`) only — no spaces or symbols like `-`, `@`, `$`.
- **Must start with a letter or underscore**, NOT a number (`1name` is invalid).
- Are **case-sensitive**: `age` and `Age` are two completely different variables.
- Cannot be a Python **keyword** (a reserved word such as `if`, `for`, `while`, `else`, `True`).

Valid names ✓

```
name = "Ali"
age = 16
my_height = 5.7
score1 = 100
```

Invalid names ✗

```
1name = "Ali" # starts with a number
my height = 5.7 # contains a space
if = 100 # keyword used
```

WORKED EXAMPLE

Using variables and case-sensitivity:

```
name = "Ali"
age = 16
my_height = 5.7
score1 = 100
print(name, age, my_height, score1)

Age = 17 # a different variable from age
print(age, Age)
```

```
OUTPUT
Ali 16 5.7 100
16 17
```

Dynamic typing — the type follows the value. `type()` tells you the type:

```
x = 5
print(x, type(x))
x = "five"
print(x, type(x))
```

```
OUTPUT
5 <class 'int'>
five <class 'str'>
```

What Python says about an invalid name:

```
1name = "Ali"
```

```
OUTPUT
File "names.py", line 1
  1name = "Ali"
    ^
SyntaxError: invalid decimal literal
```

REMEMBER IT

Choose **meaningful names**: `total_marks` is far clearer than `tm` or `x`. Use lowercase words joined by underscores (this style is called **snake_case**).

COMMON MISTAKE

`=` is **not** "equals" in the maths sense. `x = x + 1` is not a false equation; it means "take the value of `x`, add 1, and store the result back in `x`". To **compare** two values you use `==` (see 3.4.3).

اردو میں سمجھیں

ویری ایبل (Variable) میموری میں ایک نام والی جگہ ہے جس میں ڈیٹا رکھا جاتا ہے، بالکل ایک لیبل لگے ڈیے کی طرح۔ پائنتھن میں ٹائپ بتانے کی ضرورت نہیں ہوتی۔ نام حرف یا انڈرسکور سے شروع ہونا چاہیے، اس میں خالی جگہ نہیں ہو سکتی، یہ کیس سینسیٹو (Case-sensitive) ہوتا ہے اور کوئی کی ورڈ (Keyword) نہیں ہو سکتا۔

EXAM TIP

A favourite question: "Which of these variable names are valid? Give reasons." Check each name against all four rules and state the rule broken, e.g. "`2nd_place` — invalid, starts with a digit". Remember that `While` (capital W) is **valid** because keywords are lowercase and Python is case-sensitive.

CHECK YOURSELF

1. Valid or invalid? `_total`, `roll-no`, `class`, `marks2026`, `2marks`
2. Predict the output:

```
city = "Karachi"
City = "Hyderabad"
print(city)
```

Answers: 1. `_total` valid; `roll-no` invalid (hyphen); `class` invalid (keyword); `marks2026` valid; `2marks` invalid (starts with a digit). 2. `Karachi` — `city` and `City` are different variables.

3.4.1 Data Types in Python ★ Golden

LEARN IT

A **data type** specifies the **kind of data** a variable holds — a whole number, a decimal, text, True/False, or a collection of items. The type decides what you can do with the data (you can divide numbers, but not names). Python has several built-in data types:

Name	Type	Description	Example
Integer	<code>int</code>	Whole numbers without decimals (positive, negative or zero).	<code>age = 16</code>

Name	Type	Description	Example
Floating point	<code>float</code>	Numbers with a decimal point.	<code>height = 5.7</code>
String	<code>str</code>	Text data, enclosed in quotes (" " or ' ').	<code>name = "Sammy" or year = "2026"</code>
Boolean	<code>bool</code>	Logical value: True or False (capital T / F).	<code>is_student = True</code>
List	<code>list</code>	Ordered, changeable collection in square brackets [].	<code>marks = [85, 90, 75]</code>
Tuple	<code>tuple</code>	Ordered, unchangeable sequence in parentheses ().	<code>coords = (10, 20)</code>
Dictionary	<code>dict</code>	Key : value pairs in curly braces { }.	<code>student = {"name": "Ahmed", "age": 16}</code>
Set	<code>set</code>	Unordered collection of unique items (no duplicates).	<code>vowels = {'a', 'e', 'i'}</code>

Note: the slide abbreviates tuple as "tup"; the real Python type name is `tuple`, as the program below shows. The slides describe dictionaries as unordered; since Python 3.7 a dictionary remembers the order in which keys were added, but you always look items up **by key**, not by position.

WORKED EXAMPLE

Checking every type with `type()` :

```
age = 16
height = 5.7
name = "Sammy"
year = "2026"
is_student = True
marks = [85, 90, 75]
coords = (10, 20)
student = {"name": "Ahmed", "age": 16}
vowels = {'a', 'e', 'i'}

print(type(age))
print(type(height))
print(type(name), type(year))
print(type(is_student))
print(type(marks))
print(type(coords))
print(type(student))
print(type(vowels))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'> <class 'str'>
<class 'bool'>
<class 'list'>
<class 'tuple'>
<class 'dict'>
<class 'set'>
```

Notice that `year = "2026"` is a **string**, not a number, because it is inside quotes.

Working with the collection types:

```
marks = [85, 90, 75]
marks[0] = 88          # lists are changeable
marks.append(60)
print(marks)

coords = (10, 20)
print(coords[0])       # read item at position 0

student = {"name": "Ahmed", "age": 16}
print(student["name"]) # look up by key

nums = {1, 2, 2, 3, 3, 3} # duplicates are removed
print(nums)
print(len(nums))
```

OUTPUT

```
[88, 90, 75, 60]
10
Ahmed
{1, 2, 3}
3
```

Positions (called **indexes**) start at **0**, so `coords[0]` is the first item. Trying to change a tuple causes an error:

```
coords = (10, 20)
coords[0] = 50
```

OUTPUT

```
Traceback (most recent call last):
  File "tuple_test.py", line 2, in <module>
    coords[0] = 50
    ~~~~~^^^
TypeError: 'tuple' object does not support item assignment
```

REMEMBER IT

Brackets decide the collection: `[]` List (changeable), `()` Tuple (locked), `{ }` with **key:value** = Dictionary, `{ }` with single items = Set (unique). Think: a **list** is a shopping list you can edit; a **tuple** is your date of birth — it never changes.

COMMON MISTAKE

True and False must start with a capital letter. true is not a Boolean; Python treats it as an unknown variable name and gives a NameError. Also, "16" (string) and 16 (int) are different: "16" + "1" gives "161", while 16 + 1 gives 17.

اردو میں سمجھیں

ڈیٹا ٹائپ (Data Type) بتاتی ہے کہ ویری ایبل میں کس قسم کا ڈیٹا ہے۔ int مکمل اعداد، float اعشاریہ والے اعداد، str متن، اور bool صرف True یا False کے لیے ہے۔ لسٹ (List) تبدیل ہو سکتی ہے، ٹپل (Tuple) تبدیل نہیں ہو سکتا، ڈکشنری (Dictionary) کی اور ویلیو کے جوڑے ہوتے ہیں، اور سیٹ (Set) میں کوئی چیز دہرائی نہیں جاتی۔

EXAM TIP

Expect: "Name any four data types with examples" or "Differentiate list and tuple". For a difference question, use a two-column table: list — [], changeable, e.g. marks = [85, 90]; tuple — (), unchangeable, e.g. coords = (10, 20).

CHECK YOURSELF

1. What is the data type of each? 3.0, "3", 3, False, [3]
2. Predict the output: print(len({5, 5, 6, 6, 6}))
3. Which type should store a student's CNIC/B-Form number such as "42101-1234567-1"? Why?

Answers: 1. float, str, int, bool, list. 2. 2 (the set keeps only 5 and 6). 3. str, because it contains hyphens and you never do arithmetic on it.

3.4.2 Input and Output Handling

LEARN IT

Output — the print() function. A **function** is a named, ready-made piece of code that does a job when you call it with brackets. print() displays information on the screen.

- Items separated by commas are printed with one space between them.
- An **f-string** (a string with f before the quote) lets you put variables directly inside { }. It is the fastest and most readable way to format output.

Input — the input() function. It shows a message (a **prompt**), **pauses the program** and waits for the user to type something and press Enter. Whatever the user types is **always a string** by default.

Type casting means converting a value from one type to another: int() converts to an integer, float() to a decimal number, str() to a string.

```
# 1. Print only a string
print("Hello Class!")

# 2. Print a string and a variable (comma separates them)
name = "Ali"
print("Welcome,", name)

# 3. f-strings (fastest and most readable way to format)
age = 16
print(f"My name is {name} and I am {age} years old.")
```

OUTPUT

```
Hello Class!
Welcome, Ali
My name is Ali and I am 16 years old.
```

Two useful extras: `sep` changes the separator between items, and `end` changes what is printed at the end (normally a new line).

```
print("10", "09", "2026", sep="-")
print("Loading", end="...")
print("done")
print(f"Total: {7 * 3}")
```

OUTPUT

```
10-09-2026
Loading...done
Total: 21
```

WORKED EXAMPLE**Taking input with type casting.**

```
# Taking text input
user_name = input("Enter your name: ")
# Type casting: converting string input to an integer (int)
age = int(input("Enter your age: "))
# Type casting: converting string input to a decimal (float)
height = float(input("Enter your height: "))
print(f"Hello {user_name}! Next year you will be {age + 1}.")
print("Height:", height, "feet")
```

Sample run (the user types Sara, 16 and 5.4):

OUTPUT

```
Enter your name: Sara
Enter your age: 16
Enter your height: 5.4
Hello Sara! Next year you will be 17.
Height: 5.4 feet
```

Why type casting matters. Without `int()`, the numbers stay as text and `+` joins them:

```
a = input("First number: ")
b = input("Second number: ")
print(a + b)
print(int(a) + int(b))
```

OUTPUT

```
First number: 5
Second number: 3
53
8
```

"5" + "3" joins two strings into "53" (called **concatenation**). 5 + 3 adds numbers to give 8.

REMEMBER IT

"input() always gives a string." If you plan to do maths, wrap it: `int(input(...))` for whole numbers, `float(input(...))` for decimals. Read it from the inside out: first `input`, then convert.

COMMON MISTAKE

If the user types letters where a number is expected, `int()` crashes with `ValueError: invalid literal for int() with base 10` (see 3.7). Also, `int("5.4")` fails — use `float()` for decimal input.

اردو میں سمجھیں

`print()` اسکرین پر معلومات دکھاتا ہے اور `input()` صارف سے معلومات لیتا ہے۔ یاد رکھیں کہ `input()` ہمیشہ اسٹرنگ (String) دیتا ہے، اس لیے حساب کے لیے اسے `int()` یا `float()` سے تبدیل کرنا ضروری ہے۔ اسے ٹائپ کاسٹنگ (Type Casting) کہتے ہیں۔ ایف اسٹرنگ (f-string) سے ویری ایبلز کو جملے کے اندر آسانی سے لکھا جا سکتا ہے۔

CHECK YOURSELF

1. Predict the output:

```
a = "3"
b = 4
print(a * b)
print(int(a) * b)
```

2. Find the error: the user types 21 but the output is 2121, not 42.

```
num = input("Enter a number: ")
print(num * 2)
```

Answers: 1. 3333 then 12 (a string times a number repeats the string). 2. `input()` returns the string "21", and a string times 2 repeats it. Fix: `num = int(input("Enter a number: "))`.

3.4.3 Operators and Operands: Arithmetic and Assignment ★ Golden

LEARN IT

An **operand** is the value or variable being worked on. An **operator** is the symbol that performs the action. Together they form an **expression**, which Python evaluates to a single value.

In `10 + 20`: **10** is an operand, **+** is the operator, **20** is an operand, and the result is **30**.

1. **Arithmetic operators** do maths:

Operator	Purpose	Example	Result
+	Addition	5 + 3	8
-	Subtraction	10 - 4	6
*	Multiplication	4 * 2	8
/	Division (result is always a float)	9 / 2	4.5
//	Floor division (drops the decimal part)	9 // 2	4
%	Modulus (remainder)	9 % 2	1
**	Exponentiation (power)	2 ** 3	8

```

print(5 + 3)
print(10 - 4)
print(4 * 2)
print(9 / 2)
print(9 // 2)
print(9 % 2)
print(2 ** 3)
print(8 / 2)

```

OUTPUT

```

8
6
8
4.5
4
1
8
4.0

```

Notice the last line: `8 / 2` gives `4.0`, not `4`, because `/` always gives a float.

Order of operations (precedence). Python follows the same order as maths (like BODMAS): brackets first, then `**`, then `*` / `//` / `%` (left to right), then `+` / `-` (left to right). `**` is the exception: it works right to left.

```

print(2 + 3 * 4)
print((2 + 3) * 4)
print(2 ** 3 ** 2)
print(10 - 4 - 3)
print(-9 // 2)
print(17 % 5, 17 // 5)

```

OUTPUT

```

14
20
512
3
-5
2 3

```

`2 ** 3 ** 2 = 2 ** 9 = 512` (right to left). `-9 // 2` is `-5` because floor division always rounds **down** to the smaller whole number (`-4.5 → -5`).

2. Assignment operators store a value in a variable, often updating it. The short forms are called **compound assignment**.

Operator	Example	Equivalent to
<code>=</code>	<code>x = 5</code>	Assign 5 to x
<code>+=</code>	<code>x += 2</code>	<code>x = x + 2</code>
<code>-=</code>	<code>x -= 1</code>	<code>x = x - 1</code>
<code>*=</code>	<code>x *= 3</code>	<code>x = x * 3</code>
<code>/=</code> , <code>/!=</code> , <code>%=</code> , <code>**=</code>	<code>x /= 2</code>	<code>x = x / 2</code> (and so on)

WORKED EXAMPLE

Tracing assignment operators. Follow the value of `x` line by line:

```
x = 5
x += 2      # x = 5 + 2
print(x)
x -= 1      # x = 7 - 1
print(x)
x *= 3      # x = 6 * 3
print(x)
x /= 2      # x = 18 / 2
print(x)
```

OUTPUT

```
7
6
18
9.0
```

Using `%` and `//` to split digits. `% 10` gives the last digit; `// 10` removes it.

```
n = 357
last = n % 10      # remainder after dividing by 10
rest = n // 10     # drop the last digit
print("Last digit:", last)
print("Remaining:", rest)
```

OUTPUT

```
Last digit: 7
Remaining: 35
```

$357 \div 10 = 35$ remainder 7. So `357 // 10 = 35` and `357 % 10 = 7`.

REMEMBER IT

Three kinds of division: `/` = exact answer (float), `//` = quotient only ("floor"), `%` = remainder only. A number is **even** when `n % 2 == 0`.

COMMON MISTAKE

`^` is **not** power in Python — it is bitwise XOR (see bitwise operators). `2 ^ 3` gives 1, not 8. For power, always use `**`.

اردو میں سمجھیں

آپرینڈ (Operand) وہ ویلیو ہے جس پر عمل ہوتا ہے اور آپریٹر (Operator) وہ علامت ہے جو عمل کرتی ہے۔ `/` ہمیشہ اعشاریہ والا جواب دیتا ہے، `//` صرف حاصل تقسیم دیتا ہے، `%` باقی دیتا ہے اور `**` طاقت (Power) نکالتا ہے۔ `x += 2` کا مطلب ہے `x = x + 2`۔

- 2

EXAM TIP

You will be asked to evaluate expressions such as `17 // 5`, `17 % 5`, `2 ** 4` or to trace compound assignment. Show each step and write the final value with its type (e.g. `9.0` is a float, so write the `.0`).

CHECK YOURSELF

1. Predict the output:

```
x = 7
y = 2
print(x // y, x % y, x / y)
print(x ** y)
```

2. Predict the output:

```
x = 10
x += 5
x /= 4
print(x)
```

Answers: 1. 3 1 3.5 then 49 . 2. 3 (10 + 5 = 15; 15 // 4 = 3).

3.4.3 Relational, Logical and Membership Operators ★ Golden

LEARN IT

3. Relational (comparison) operators compare two values and return a **Boolean** (True or False). They are used in conditions.

Op	Description	Example (a = 10, b = 5)	Result
=	Equal to	a == b	False
≠	Not equal to	a != b	True
>	Greater than	a > b	True
<	Less than	a < b	False
≥	Greater than or equal to	a ≥ 10	True
≤	Less than or equal to	a ≤ b	False

```
a = 10
b = 5
print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a ≥ 10)
print(a ≤ b)
```

OUTPUT

```
False
True
True
False
True
False
```

4. Logical operators combine several conditions:

A	B	A and B	A or B	not A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

- `and` — True **only if both** conditions are true.
- `or` — True if **at least one** condition is true.
- `not` — **reverses** the result (True becomes False, False becomes True).

```
a = 10
b = 5
print(a > 5 and b > 5)
print(a > 5 or b > 5)
print(not a > 5)
```

OUTPUT

```
False
True
False
```

5. Membership operators check whether a value exists inside a sequence (such as a string or list): `in` and `not in`.

```
name = "Python"
print("P" in name)
print("z" not in name)
print("p" in name)
marks = [85, 90, 75]
print(90 in marks)
```

OUTPUT

```
True
True
False
True
```

"p" in name is False because strings are case-sensitive: the word has a capital P.

WORKED EXAMPLE

Real-life conditions. A person can apply for a CNIC if they are 18 or older **and** have the documents. Marks are invalid if they are below 0 **or** above 100.

```
age = 20
has_cnic = True
print(age ≥ 18 and has_cnic)
marks = 45
print(marks < 0 or marks > 100)
```

OUTPUT

```
True
False
```

Step by step: $20 \geq 18$ is True, has_cnic is True, True and True = **True**. For marks: $45 < 0$ is False, $45 > 100$ is False, False or False = **False** (the marks are valid).

REMEMBER IT

AND is strict, OR is generous. AND is like a teacher who needs both homework and attendance; OR is like a shop that accepts cash **or** card. Order of evaluation: **not** first, then **and**, then **or**.

COMMON MISTAKE

Do not confuse `=` (assignment, "store") with `==` (comparison, "is it equal?"). Writing `if x = 5:` is a **syntax error**; the correct form is `if x == 5:`.

اردو میں سمجھیں

ریشنل آپریٹرز (Relational Operators) دو ویلیوز کا موازنہ کر کے True یا False دیتے ہیں۔ لاجیکل آپریٹرز میں `and` تب True دیتا ہے جب دونوں شرطیں درست ہوں، `or` تب جب کم از کم ایک درست ہو، اور `not` نتیجہ الٹ دیتا ہے۔ `in` اور `not in` دیکھتے ہیں کہ کوئی چیز اسٹرنگ یا لسٹ میں موجود ہے یا نہیں۔

EXAM TIP

Typical: "Evaluate `a > b` and `b > 3` for `a = 10, b = 5`" or "Write the truth table of `and` / `or`". Write each sub-condition's value first, then combine. Remember: `==` compares, `=` assigns.

CHECK YOURSELF

1. Predict the output:

```
print(True and False or True)
print(not (5 > 3) or 2 == 2.0)
```

2. Find the error:

```
x = 5
if x = 5:
    print("five")
```

Answers: 1. True and True. (True and False = False; False or True = True. not True = False; 2 == 2.0 is True; False or True = True.) 2. = is assignment; a comparison needs ==. Python reports SyntaxError: invalid syntax. Maybe you meant '=' or ':=' instead of '='? Fix: `if x == 5:`

3.4.3 Bitwise Operators ★ Golden

LEARN IT

6. Bitwise operators treat numbers as **binary digits** (bits: 0s and 1s) and work on them **bit by bit**, column by column. Think of each bit as a switch that is ON (1) or OFF (0). Python's `bin()` function shows the binary form of a number (it starts with `0b`).

- **AND (&)** — gives 1 **only if both** bits are 1, otherwise 0.
- **OR (|)** — gives 1 if **at least one** bit is 1.
- **XOR (^)** — gives 1 if the bits are **different**, 0 if they are the **same**.
- **NOT (~)** — inverts all bits (`0 → 1, 1 → 0`). In Python the result is the number made negative, minus 1: `~x = -(x + 1)`. So `~10` gives `-11`.
- **Left shift (<<)** — moves bits left and adds 0s on the right. Each shift **multiplies by 2**.
- **Right shift (>>)** — moves bits right and removes the rightmost bits. Each shift **divides by 2** (floor division).

Bit A	Bit B	A & B	A B	A ^ B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

WORKED EXAMPLE

Worked example: a = 10, b = 6. First convert to binary using place values 8, 4, 2, 1: 10 = 8 + 2 = **1010**; 6 = 4 + 2 = **0110**. Then work column by column.

	8	4	2	1	Decimal
a = 10	1	0	1	0	10
b = 6	0	1	1	0	6
a & b	0	0	1	0	2
a b	1	1	1	0	14
a ^ b	1	1	0	0	12

- AND: only the "2" column has 1 in both → 0010 = **2**.
- OR: columns 8, 4 and 2 have at least one 1 → 1110 = 8 + 4 + 2 = **14**.
- XOR: columns 8 and 4 are different; column 2 is the same (both 1) → 1100 = 8 + 4 = **12**.
- NOT: $\sim 10 = -(10 + 1) = \mathbf{-11}$.

```
a = 10
b = 6
print(bin(a), bin(b))
print(a & b)
print(a | b)
print(a ^ b)
print(~a)
```

OUTPUT

```
0b1010 0b110
2
14
12
-11
```

Worked example: shifts. 5 in 8 bits is **0000 0101**.

Expression	Binary	Decimal	What happened
5	0000 0101	5	start
5 << 1	0000 1010	10	add one 0 on the right (5 × 2)
5 << 2	0001 0100	20	add 00 on the right (5 × 4)
20 >> 1	0000 1010	10	remove the rightmost bit (20 ÷ 2)
20 >> 2	0000 0101	5	remove two bits (20 ÷ 4)

```

a = 5          # Binary: 0000 0101
print(a << 1)   # 0000 1010
print(a << 2)   # 0001 0100
b = 20         # Binary: 0001 0100
print(b >> 1)   # 0000 1010
print(b >> 2)   # 0000 0101
print(bin(20 >> 2))

```

OUTPUT

```

10
20
10
5
0b101

```

REMEMBER IT

Shortcuts: $x \ll n = x \times 2^n$, $x \gg n = x // 2^n$, $\sim x = -x - 1$. For AND/OR/XOR always write both numbers in binary, one under the other, with the same number of bits.

COMMON MISTAKE

Do not mix up the logical and bitwise operators: `and` / `or` / `not` work on True/False conditions, while `&` / `|` / `~` work on the bits of integers. Also remember `bin(6)` prints `0b110`, without the leading zero — add zeros yourself when lining up columns.

اردو میں سمجھیں

بٹ وائز آپریٹرز (Bitwise Operators) اعداد کو بائری میں بدل کر ہر بٹ پر الگ الگ کام کرتے ہیں۔ `&` تب 1 دیتا ہے جب دونوں بٹ 1 ہوں، `|` تب جب کوئی ایک بھی 1 ہو، اور `^` تب جب دونوں بٹ مختلف ہوں۔ `~` تمام بٹ الٹ دیتا ہے، اس لیے `~10` کا جواب -11 ہے۔ لیفٹ شفٹ عدد کو 2 سے ضرب دیتا ہے اور رائٹ شفٹ 2 سے تقسیم کرتا ہے۔

EXAM TIP

Exams ask you to "evaluate `12 & 10`, `12 | 10`, `12 ^ 10`, `~5`, `7 >> 1` showing binary working". Marks are given for the binary conversion and column working, not just the final number. Always draw the column table.

CHECK YOURSELF

1. Predict the output (show the binary working):

```

print(12 & 10, 12 | 10, 12 ^ 10)
print(~5, 7 >> 1, 3 << 3)

```

2. What does `bin(13)` print?

Answers: 1. 8 14 6 and -6 3 24. $12 = 1100$, $10 = 1010$: AND = 1000 = 8, OR = 1110 = 14, XOR = 0110 = 6. $\sim 5 = -(5 + 1) = -6$. $7 = 111 \rightarrow 11 = 3$. $3 = 11 \rightarrow 11000 = 24$ (3×8). 2. `0b1101` ($8 + 4 + 1$).

3.5 Control Structures ★ Golden

LEARN IT

Control structures decide the **order in which statements are executed** in a program (the "flow of control"). Almost every program is built from just three basic structures:

1. **Sequence** — do the steps one after another, top to bottom.
2. **Selection** — choose a path depending on a condition (`if`, `else`).
3. **Repetition** — repeat steps (loops: `for`, `while`).

WORKED EXAMPLE

Morning routine as control structures:

- Sequence: wake up → brush teeth → have breakfast.
- Selection: **if** it is raining, take an umbrella; **else** take a cap.
- Repetition: **while** the bus has not come, keep waiting.

REMEMBER IT

Remember "**S-S-R**": Sequence, Selection, Repetition — "Step, Select, Repeat".

COMMON MISTAKE

Selection and repetition both use conditions, but they are different: selection runs a block **at most once**; repetition can run a block **many times**.

اردو میں سمجھیں

کنٹرول اسٹرکچرز (Control Structures) طے کرتے ہیں کہ پروگرام کی ہدایات کس ترتیب سے چلیں گی۔ تین بنیادی اسٹرکچر ہیں: سیکوئنس (ترتیب وار)، سلیکشن (شرط کی بنیاد پر انتخاب) اور ریپیٹیشن (دہرانا یعنی لوپ)۔

EXAM TIP

"Name and explain the three basic control structures" is a very common short question. Give a one-line definition and a two-line Python example for each.

CHECK YOURSELF

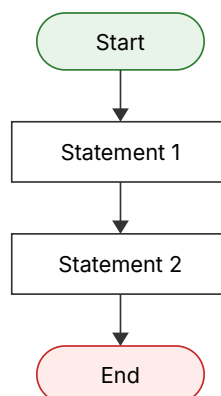
1. Name the three basic control structures.
2. Which structure is used for "keep asking for the password until it is correct"?

Answers: 1. Sequence, selection, repetition. 2. Repetition (a while loop).

3.5.1 Sequence

LEARN IT

Sequence means statements are executed **one after another, strictly from top to bottom**, in the exact order they are written. This is the **default behaviour** of Python. No statement is skipped or repeated.



Sequence: control flows straight down from one statement to the next.

WORKED EXAMPLE

Input → Process → Output in sequence.

```
print("Enter two numbers")
a = int(input("Enter first number: ")) # Step 1: Input
b = int(input("Enter second number: ")) # Step 2: Input
total = a + b # Step 3: Process
print("Sum =", total) # Step 4: Output
```

Sample run (user types 7 and 5):

OUTPUT

```
Enter two numbers
Enter first number: 7
Enter second number: 5
Sum = 12
```

Each line runs exactly once, in order. If you moved line 4 (`total = a + b`) above the input lines, Python would stop with a `NameError` because `a` and `b` would not exist yet.

REMEMBER IT

Sequence is like reading a page: line 1, then line 2, then line 3. Most programs follow the pattern **Input → Process → Output**.

COMMON MISTAKE

Order matters. A variable must be given a value **before** it is used. Using it first causes `NameError: name '...' is not defined`.

اردو میں سمجھیں

سیکوانس (Sequence) کا مطلب ہے کہ ہدایات بالکل اسی ترتیب سے ایک کے بعد ایک اوپر سے نیچے چلتی ہیں جس ترتیب سے لکھی گئی ہیں۔ یہ پائنتن کا عام طریقہ ہے۔ کسی ویری ایبل کو استعمال کرنے سے پہلے اس میں ویلیو رکھنا ضروری ہے۔

CHECK YOURSELF

1. Predict the output:

```
x = 4
y = x * 2
x = 10
print(x, y)
```

2. What is the default control structure in Python?

Answers: 1. 10 8 — `y` was calculated when `x` was still 4; changing `x` later does not change `y`. 2. Sequence.

3.5.2 Selection Statements: if and if-else ★ Golden

LEARN IT

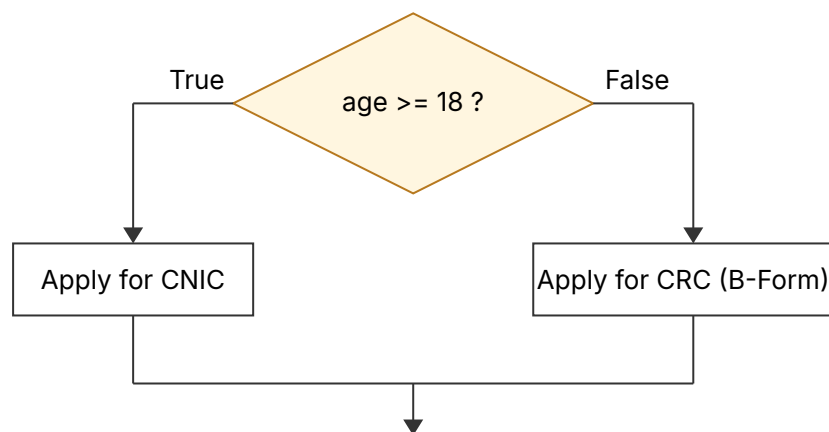
Selection changes the flow of a program based on a **condition** — an expression that is either True or False. It helps a program respond intelligently to different situations.

1. The if statement executes a block of code **only if** the condition is True. If the condition is False, the block is completely skipped.

```
if condition:
    statement(s)    # runs only when condition is True
```

2. The if-else statement provides **two paths**: if the condition is true, do this; **else** (otherwise), do that. Exactly one of the two blocks always runs.

```
if condition:
    block A        # condition True
else:
    block B        # condition False
```



An if-else statement: one diamond (condition), two paths, then the paths join again.

WORKED EXAMPLE

The if statement.

```
age = int(input("Enter your age: "))
if age >= 18:
    print("You are an Adult.")    # runs only if age >= 18
print("Program continues...")    # runs every time
```

Run 1 (user types 20):

```
OUTPUT
Enter your age: 20
You are an Adult.
Program continues...
```

Run 2 (user types 15):

```
OUTPUT
Enter your age: 15
Program continues...
```

The last line is not indented, so it is outside the `if` block and runs every time.

The if-else statement.

```
age = int(input("Enter your age: "))
if age ≥ 18:
    print("Apply for CNIC.")
else:
    print("Apply for CRC (B-Form).")
```

Sample run (user types 16):

```
OUTPUT
Enter your age: 16
Apply for CRC (B-Form).
```

Even or odd — a classic if-else program:

```
n = 7
if n % 2 == 0:
    print(n, "is even")
else:
    print(n, "is odd")
```

```
OUTPUT
7 is odd
```

7 % 2 = 1, and 1 == 0 is False, so the `else` block runs.

REMEMBER IT

Formatting rule: put a **colon (:)** after `if` and `else`, and **indent** the code inside the blocks. `else` never has a condition of its own.

COMMON MISTAKE

Common mistakes: forgetting the colon (`SyntaxError: expected ':'`), writing a condition after `else` (`else age < 18: is wrong`), and not indenting `else` at the same level as its `if`.

اردو میں سمجھیں

سلیکشن (Selection) میں پروگرام شرط کی بنیاد پر فیصلہ کرتا ہے۔ `if` کا بلاک صرف تب چلتا ہے جب شرط درست (True) ہو۔ `if-else` میں دو راستے ہوتے ہیں: شرط درست ہو تو پہلا بلاک، ورنہ `else` والا بلاک چلتا ہے۔ `if` اور `else` کے بعد کولن (:) اور اندر کے کوڈ کی انڈینٹیشن ضروری ہے۔

EXAM TIP

Asked as: "Write a program that inputs a number and prints whether it is even or odd / positive or negative / pass or fail." Write the full program with `int(input())`, the condition, both blocks, and a sample run.

CHECK YOURSELF

1. Predict the output when `marks = 32`:

```
marks = 32
if marks ≥ 33:
    print("Pass")
else:
    print("Fail")
print("Done")
```

2. Find the two errors:

```
n = 5
if n > 0
    print("Positive")
else n ≤ 0:
    print("Not positive")
```

Answers: 1. Fail then Done . 2. Missing colon after `if n > 0`; `else` must not have a condition — write `else:` .

3.5.2 Selection Statements: if-elif-else and Nested if ★ Golden

LEARN IT

3. The **if-elif-else** statement handles **many conditions**. (`elif` is short for "else if".) Python checks the conditions **from top to bottom**. As soon as it finds a True condition, it runs that block and **ignores all the rest**. If none is True, the `else` block runs.

4. **Nested if** is an `if` inside another `if`. It is used when a decision needs to pass **several levels of checks**: the inner check happens only if the outer check passed.

WORKED EXAMPLE

Grading with if-elif-else.

```
marks = int(input("Enter marks: "))
if marks ≥ 80:
    print("A-1 Grade")
elif marks ≥ 70:
    print("A Grade")
elif marks ≥ 60:
    print("B Grade")
else:
    print("Fail or below B")
```

Sample run (user types 75):

OUTPUT

```
Enter marks: 75
A Grade
```

Trace for different marks:

marks	>= 80?	>= 70?	>= 60?	Output
92	True	(not checked)	(not checked)	A-1 Grade
75	False	True	(not checked)	A Grade
60	False	False	True	B Grade
41	False	False	False	Fail or below B

Loan approval with nested if.

```
age = 20
salary = 35000
if age >= 18:
    # passed first check
    if salary >= 30000:
        # passed second check
        print("Loan APPROVED!")
    else:
        print("Salary too low for loan.")
else:
    print("Too young for loan.")
```

OUTPUT

Loan APPROVED!

age	salary	Outer check (age >= 18)	Inner check (salary >= 30000)	Output
20	35000	True	True	Loan APPROVED!
20	25000	True	False	Salary too low for loan.
16	35000	False	(never checked)	Too young for loan.

When you only need a yes/no answer, two levels can often be joined with `and` — but then you lose the separate messages:

```
age = 20
salary = 25000
if age >= 18 and salary >= 30000:
    print("Loan APPROVED!")
else:
    print("Loan not approved.")
```

OUTPUT

Loan not approved.

REMEMBER IT

In an `if-elif-else` ladder, put the **strictest condition first** (highest marks first). Think of a ladder: you step off at the first rung that fits.

COMMON MISTAKE

Wrong order gives wrong answers. Here 85 marks gets "B Grade", because $85 \geq 60$ is True first and the other conditions are ignored:

```
marks = 85
if marks ≥ 60:
    print("B Grade")
elif marks ≥ 70:
    print("A Grade")
elif marks ≥ 80:
    print("A-1 Grade")
```

OUTPUT
B Grade

This is a **logical error**: no crash, just a wrong result. Fix it by checking ≥ 80 first.

اردو میں سمجھیں

if-elif-else میں کئی شرطیں اوپر سے نیچے ترتیب سے دیکھی جاتی ہیں، اور جو شرط سب سے پہلے درست ہو اسی کا بلاک چلتا ہے، باقی سب چھوڑ دی جاتی ہیں۔ نیسٹڈ ایف (Nested if) میں ایک if دوسرے if کے اندر ہوتا ہے، اور اندر والی شرط تب ہی دیکھی جاتی ہے جب باہر والی شرط درست ہو۔

EXAM TIP

Grade-calculator programs (if-elif-else) and "largest of three numbers" are very common long questions. Check conditions in the right order, indent nested blocks carefully (8 spaces for the inner block), and give a sample run. Tracing tables like the ones above earn full marks in "predict the output" questions.

CHECK YOURSELF

1. Predict the output:

```
temp = 30
if temp > 35:
    print("Hot")
elif temp > 25:
    print("Warm")
elif temp > 15:
    print("Pleasant")
else:
    print("Cold")
```

2. Predict the output:

```
x = 15
if x > 10:
    if x > 20:
        print("A")
    else:
        print("B")
else:
    print("C")
```

Answers: 1. Warm (30 > 35 is False; 30 > 25 is True, so the rest are skipped even though 30 > 15 is also True). 2. B (outer check 15 > 10 passes; inner check 15 > 20 fails).

3.5.3 Repetition Statements: the for Loop and range() ★ Golden

LEARN IT

A **repetition statement (loop)** lets a block of code run **again and again**. This greatly reduces the need to write the same code many times. Each single run of the loop body is called an **iteration**.

1. The for loop is used when the **exact number of iterations is known**. It takes each value from a sequence one by one and stores it in the **loop variable** (e.g. `i`).

The `range()` function generates the sequence of numbers for the loop:

Syntax	How it works	Example	Values produced
<code>range(stop)</code>	From 0 up to, but not including , stop.	<code>range(3)</code>	0 1 2
<code>range(start, stop)</code>	From start up to, but not including, stop.	<code>range(2, 5)</code>	2 3 4
<code>range(start, stop, step)</code>	From start towards stop, jumping by step.	<code>range(1, 6, 2)</code>	1 3 5

```
for i in range(3):  
    print(i)
```

OUTPUT

```
0  
1  
2
```

```
for i in range(1, 6, 2):  
    print(i)
```

OUTPUT

```
1  
3  
5
```

```
for i in range(2, 5):  
    print(i)
```

OUTPUT

```
2  
3  
4
```

```
for i in range(0, 6, 2):  
    print("Even:", i)
```

OUTPUT

```
Even: 0  
Even: 2  
Even: 4
```

You can see all the values of a range at once by converting it to a list. A negative step counts **down**; if start equals stop, the range is empty:

```
print(list(range(5)))  
print(list(range(2, 5)))  
print(list(range(1, 6, 2)))  
print(list(range(10, 0, -2)))  
print(list(range(5, 5)))
```

OUTPUT

```
[0, 1, 2, 3, 4]  
[2, 3, 4]  
[1, 3, 5]  
[10, 8, 6, 4, 2]  
[]
```

WORKED EXAMPLE

Multiplication table of 7:

```
for i in range(1, 6):  
    print(f"7 x {i} = {7 * i}")
```

```
OUTPUT  
7 x 1 = 7  
7 x 2 = 14  
7 x 3 = 21  
7 x 4 = 28  
7 x 5 = 35
```

Adding numbers with an accumulator. An **accumulator** is a variable that starts at 0 and collects a running total.

```
total = 0  
for i in range(1, 11):  
    total += i  
print("Sum of 1 to 10 =", total)
```

```
OUTPUT  
Sum of 1 to 10 = 55
```

i	1	2	3	4	5	...	10
total after	1	3	6	10	15	...	55

Looping over a string or list. A `for` loop can also take items directly from any sequence:

```
for ch in "Sindh":  
    print(ch, end=" ")  
print()  
for m in [85, 90, 75]:  
    print(m + 5)
```

```
OUTPUT  
S i n d h  
90  
95  
80
```

REMEMBER IT

range stops one short: `range(1, 11)` gives 1 to 10. To include a number `n`, write `range(..., n + 1)`. Number of values in `range(a, b) = b - a`.

COMMON MISTAKE

Writing `range(1, 10)` to print 1 to 10 misses 10 — an "off-by-one" logical error. Also, the loop body must be indented; a `for` line without an indented block gives `IndentationError: expected an indented block after 'for' statement`.

اردو میں سمجھیں

لوپ (Loop) کوڈ کے ایک بلاک کو بار بار چلاتا ہے۔ `for` لوپ تب استعمال ہوتا ہے جب پہلے سے معلوم ہو کہ کتنی بار دہرانا ہے۔ `range(stop)` صفر سے شروع ہوتا ہے، `range(start, stop)` سے، اور `range(start, stop, step)` ہر بار `step` کے برابر چھلانگ لگاتا ہے۔ آخری عدد (`stop`) کبھی شامل نہیں ہوتا۔

EXAM TIP

Expect "predict the output" questions on `range()` with a step, and programs such as "print a table", "sum of first n numbers", "even numbers from 1 to 20". Always state what range produces before tracing.

CHECK YOURSELF

1. Predict the output:

```
for i in range(10, 0, -3):  
    print(i, end=" ")
```

2. Predict the output:

```
s = 0  
for k in range(2, 11, 2):  
    s += k  
print(s)
```

3. Find the error:

```
for i in range(3):  
print(i)
```

Answers: 1. 10 7 4 1. 2. 30 (2 + 4 + 6 + 8 + 10). 3. The body is not indented: `IndentationError`. Indent `print(i)` by 4 spaces.

3.5.3 The while Loop, break and continue ★ Golden

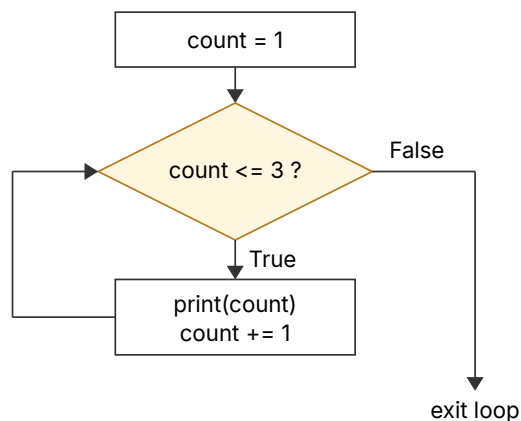
LEARN IT

2. The while loop repeats a block of code **as long as a condition remains True**. It is used when the exact number of iterations is **not known in advance** (for example, "keep asking until the password is correct").

A while loop needs three parts: (1) **initialise** a variable before the loop, (2) **test** the condition, (3) **update** the variable inside the loop. If you forget the update, the condition never becomes False and the loop never stops — an **infinite loop**.

Loop control statements change the normal flow of a loop:

- **break** — exits (stops) the loop **entirely and immediately**.
- **continue** — **skips the rest of the current iteration** and jumps to the next one.



A while loop: test the condition, run the body, go back and test again.

```
count = 1
while count ≤ 3:
    print(count)
    count += 1 # VERY IMPORTANT: update the condition!
```

OUTPUT

```
1
2
3
```

WORKED EXAMPLE

Trace of the while loop above:

count	count ≤ 3?	Printed	count after update
1	True	1	2
2	True	2	3
3	True	3	4
4	False	—	loop ends

Sum of digits — we do not know how many digits the number has, so `while` fits well:

```
n = 4072
total = 0
while n > 0:
    digit = n % 10
    total += digit
    n = n // 10
print("Sum of digits =", total)
```

OUTPUT

```
Sum of digits = 13
```

Digits taken: 2, 7, 0, 4 → 2 + 7 + 0 + 4 = 13. `n` becomes 407, 40, 4, 0, and then the loop stops.

Password check — repeat until the user gets it right:

```
password = input("Enter password:")
while password ≠ "sindh123":
    print("Wrong password, try again.")
    password = input("Enter password: ")
print("Welcome!")
```

Sample run (user types `abc`, `sindh`, `sindh123`):

OUTPUT

```
Enter password: abc
Wrong password, try again.
Enter password: sindh
Wrong password, try again.
Enter password: sindh123
Welcome!
```

break and continue side by side:

```
for i in range(5):
    if i == 3:
        break
    print(i)
print("Loop ended")
```

```
OUTPUT
0
1
2
Loop ended
```

At $i = 3$ the loop stops completely, so 3 and 4 are never printed.

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

```
OUTPUT
0
1
2
4
```

At $i = 3$ only that iteration is skipped; the loop carries on with 4.

while True with break — a loop that runs "forever" until **break** stops it:

```
n = 1
while True:
    if n * n > 50:
        break
    n += 1
print("First number whose square is more than 50:", n)
```

```
OUTPUT
First number whose square is more than 50: 8
```

REMEMBER IT

break = "exit the room"; **continue** = "skip this turn". For every **while** loop, check the **I-T-U** rule: Initialise, Test, Update.

COMMON MISTAKE

Infinite loop — find the error:

```
count = 1
while count ≤ 5:
    print(count)
```

count is never updated, so $\text{count} \leq 5$ stays True and the program prints 1 forever. Fix: add **count += 1** inside the loop. (To stop a running program in the terminal, press **Ctrl + C**.)

اردو میں سمجھیں

while لوپ اس وقت تک چلتا رہتا ہے جب تک شرط درست (True) رہے، اور یہ تب استعمال ہوتا ہے جب پہلے سے معلوم نہ ہو کہ کتنی بار دہرانا ہے۔ لوپ کے اندر ویری ایبل کو اپڈیٹ کرنا لازمی ہے ورنہ لوپ کبھی ختم نہیں ہوگا (Infinite Loop)۔ **break** لوپ کو فوراً مکمل طور پر روک دیتا ہے، جبکہ **continue** صرف موجودہ چکر چھوڑ کر اگلے چکر پر چلا جاتا ہے۔

EXAM TIP

"Differentiate **break** and **continue** with examples" and "What is an infinite loop?" are frequent. Use the two side-by-side programs above with their outputs — examiners like seeing the difference in the output (0 1 2 vs 0 1 2 4).

CHECK YOURSELF

1. Predict the output:

```
n = 1
while n < 20:
    n = n * 3
print(n)
```

2. Predict the output:

```
for i in range(1, 8):
    if i % 3 == 0:
        continue
    if i == 7:
        break
    print(i, end=" ")
```

Answers: 1. 27 (n: 1 → 3 → 9 → 27; 27 < 20 is False, so the loop stops). 2. 1 2 4 5 (3 and 6 are skipped by `continue`; at 7 the loop breaks).

3.5.3 Nested Loops and for vs while ★ Golden

LEARN IT

A **nested loop** is a loop inside another loop. The **inner loop runs completely, from start to finish, for every single iteration of the outer loop**. If the outer loop runs 2 times and the inner loop runs 3 times, the inner body runs $2 \times 3 = 6$ times.

```
for outer in range(1, 3):          # runs 2 times (1, 2)
    for inner in range(1, 4):      # runs 3 times per outer (1, 2, 3)
        print(f"Outer: {outer}, Inner: {inner}")
```

OUTPUT

```
Outer: 1, Inner: 1
Outer: 1, Inner: 2
Outer: 1, Inner: 3
Outer: 2, Inner: 1
Outer: 2, Inner: 2
Outer: 2, Inner: 3
```

WORKED EXAMPLE

A star triangle. The outer loop chooses the row; the inner loop prints that many stars on the same line (`end=""`), and the empty `print()` moves to the next line.

```
for row in range(1, 5):
    for col in range(row):
        print("*", end="")
    print()
```

OUTPUT

```
*
**
***
****
```

Row 1: inner loop runs once → *. Row 2: twice → **. And so on up to row 4.

for loop vs while loop

for loop	while loop
Used when the number of iterations is known .	Used when the number of iterations is unknown (condition-based).
Automatically updates the loop variable.	Requires a manual update of the counter (risk of an infinite loop).
Cleaner and shorter for counting tasks.	Better for dynamic situations (like "keep playing until the user says Stop").
<code>for i in range(1, 4): print(i)</code>	<code>i = 1 / while i ≤ 3: / print(i) / i += 1</code>

REMEMBER IT

Ask yourself: "Do I know **how many times**?" Yes → `for`. "Do I only know **when to stop**?" → `while`. Nested loops are like a clock: the minute hand (inner loop) goes all the way round for each step of the hour hand (outer loop).

COMMON MISTAKE

Indentation decides which loop a line belongs to. In the triangle program, if `print()` were indented under the inner loop, every star would go on its own line.

اردو میں سمجھیں

نیسٹڈ لوپ (Nested Loop) میں ایک لوپ دوسرے لوپ کے اندر ہوتا ہے، اور باہر والے لوپ کے ہر چکر میں اندر والا لوپ شروع سے آخر تک مکمل چلتا ہے۔ جب دہرائے کی تعداد معلوم ہو تو `for` لوپ، اور جب صرف رکنے کی شرط معلوم ہو تو `while` لوپ بہتر ہے۔

EXAM TIP

"Differentiate between for and while loops" (give three points and an example of each) and "How many times will the inner statement run?" (multiply the counts) are typical. For pattern programs, explain what the outer and inner loops each control.

CHECK YOURSELF

1. Predict the output:

```
count = 0
for i in range(3):
    for j in range(2):
        count += 1
print(count)
```

2. Which loop would you use to keep reading numbers until the user enters 0?

Answers: 1. 6 (3 × 2). 2. A `while` loop, because the number of inputs is not known in advance.

Practical Activity: Shopping Bill Calculator

LEARN IT

This program combines all three control structures: **sequence** (steps in order), **selection** (checking "done", checking the price, checking for a discount) and **repetition** (a `while` loop that keeps adding items). It uses a **flag** — a Boolean variable (`shopping`) that controls when the loop stops. `item.lower()` converts the text to lowercase so that "done", "Done" and "DONE" all work.

```

print("==== SHOPPING BILL ====")
total = 0.0
count = 0
shopping = True

# REPETITION: keep adding items
while shopping:
    item = input("Enter Item Name (or 'done' to stop): ")
    # SELECTION: check if user wants to stop
    if item.lower() == "done":
        shopping = False
    else:
        price = float(input(f"Enter Price of {item}: "))
        # SELECTION: validate price
        if price <= 0:
            print("Invalid price!")
        else:
            total += price
            count += 1

# Summary and discount
if total >= 5000:
    discount = total * 0.20 # 20% off
else:
    discount = 0

final = total - discount
print(f"Total Items: {count}")
print(f"Total Bill: {total}")
print(f"Final to Pay: {final}")

```

WORKED EXAMPLE

Sample run. The user buys Rice for 3000, types a wrong price -50 for Soap, buys Oil for 2500, then types DONE :

OUTPUT

```

==== SHOPPING BILL ====
Enter Item Name (or 'done' to stop): Rice
Enter Price of Rice: 3000
Enter Item Name (or 'done' to stop): Soap
Enter Price of Soap: -50
Invalid price!
Enter Item Name (or 'done' to stop): Oil
Enter Price of Oil: 2500
Enter Item Name (or 'done' to stop): DONE
Total Items: 2
Total Bill: 5500.0
Final to Pay: 4400.0

```

Step	item	price	total	count
1	Rice	3000.0	3000.0	1
2	Soap	-50.0 (invalid, not added)	3000.0	1
3	Oil	2500.0	5500.0	2
4	DONE → shopping = False	—	5500.0	2

After the loop: $5500.0 \geq 5000$, so $\text{discount} = 5500.0 \times 0.20 = 1100.0$, and $\text{final} = 5500.0 - 1100.0 = \mathbf{4400.0}$.

REMEMBER IT

Label the parts of any program with comments, as here: `# REPETITION`, `# SELECTION`. It helps you, your teacher and the examiner see the structure.

COMMON MISTAKE

`total` and `count` must be set to 0 **before** the loop. If you put `total = 0.0` inside the loop, it resets on every item and the bill is wrong.

اردو میں سمجھیں

یہ شاپنگ بل پروگرام تینوں کنٹرول اسٹرکچرز کو ملا کر بنایا گیا ہے: `while` لوپ سے اشیاء بار بار شامل ہوتی ہیں، `if-else` سے غلط قیمت روکی جاتی ہے اور 5000 یا زیادہ کے بل پر 20 فیصد رعایت دی جاتی ہے۔

CHECK YOURSELF

1. What will "Final to Pay" be if the user buys one item for 4000 and then types `done` ?
2. What is the purpose of the variable `shopping` ?

Answers: 1. Final to Pay: 4000.0 (4000 is less than 5000, so no discount). 2. It is a flag that keeps the `while` loop running; setting it to `False` stops the loop.

3.6 Libraries in Python

LEARN IT

A **library** (also called a **module**) is a collection of pre-written code that offers ready-made functions. Instead of writing a square-root algorithm from scratch, you import `math` and call `math.sqrt()`. Libraries save time, reduce bugs and let you stand on other programmers' work.

There are five common ways to import a library. Method 3 (import only what you need) is the best habit: it is clearer and uses less memory than `from math import *`.

Method	Syntax	How you call the function
1. Entire module	<code>import math</code>	<code>math.sqrt(25)</code>
2. Alias (nickname)	<code>import math as m</code>	<code>m.sqrt(16)</code>
3. Specific function	<code>from math import sqrt</code>	<code>sqrt(36)</code>
4. Several functions	<code>from math import sqrt, pi</code>	<code>sqrt(9)</code> , <code>pi</code>
5. Everything (*)	<code>from math import *</code>	<code>sqrt(64)</code> — avoid this

WORKED EXAMPLE

Five imports, one result each:

```
import math
print(math.sqrt(25))
import math as m
print(m.sqrt(16))
from math import sqrt
print(sqrt(36))
from math import sqrt, pi
print(pi)
from math import *
print(sqrt(64))
```

OUTPUT

```
5.0
4.0
6.0
3.141592653589793
8.0
```

COMMON MISTAKE

`from math import *` dumps every name into your program. If you later write a function called `sqrt`, you silently overwrite the library one. Prefer methods 1–4.

اردو میں سمجھیں

لائبریری پہلے سے لکھا ہوا کوڈ ہے جسے `import` سے پروگرام میں لایا جاتا ہے۔ سب سے اچھا طریقہ یہ ہے کہ صرف وہ فنکشن درآمد کریں جس کی ضرورت ہو، ستارہ `*` سے سب کچھ نہ کھینچیں۔

CHECK YOURSELF

1. Write the line that lets you call `ceil` without writing `math.` in front.
2. Why is an alias such as `import pandas as pd` useful?

Answers: 1. `from math import ceil`. 2. The library name is long; a short nickname keeps later lines short and is a community convention.

3.6.1 Built-in Libraries

LEARN IT

Built-in libraries come pre-installed with Python. You only import them; you do not run `pip`. Three that every Class XI student must know are `math`, `random` and `datetime`.

math

```
import math
print(math.pi)
print(math.ceil(4.3))
print(math.floor(4.9))
print(math.pow(2, 4))
print(math.sqrt(81))
```

OUTPUT

```
3.141592653589793
5
4
16.0
9.0
```

random and datetime

```
import random
print(random.randint(1, 1))
print(0.0 ≤ random.random() < 1.0)

import datetime
now = datetime.datetime(2026, 9, 27, 11, 59, 40)
print(now.strftime("%d-%m-%Y"))
```

OUTPUT

```
1
True
27-09-2026
```

`randint(a, b)` includes both ends. `random()` is a float in `[0.0, 1.0)`. `strftime` formats a date; `%d-%m-%Y` is day-month-year.

REMEMBER IT

`ceil` always rounds **up** (ceiling); `floor` always rounds **down**. `round()` (built-in, not in `math`) goes to the nearest integer and is a different function.

اردو میں سمجھیں

`math` حساب کے لیے، `random` بے ترتیب عدد کے لیے اور `datetime` تاریخ وقت کے لیے بنی ہوئی لائبریریاں ہیں۔ انہیں انسٹال نہیں کرنا پڑتا، صرف `import` کافی ہے۔

CHECK YOURSELF

1. What does `math.ceil(4.0)` return?
2. Write a line that prints a random integer from 1 to 6 (a dice).

Answers: 1. 4 (already an integer). 2. `print(random.randint(1, 6))` after `import random`.

3.6.2 Third-Party Libraries

LEARN IT

Third-party libraries are written by the community. They are **not** bundled with Python, so you install them from a terminal with `pip install library_name` and then import them as usual.

Library	Primary use
NumPy	Heavy numerical computing and large arrays
Pandas	Data analysis; table-like data (CSV / Excel)
Matplotlib	Data visualisation — graphs and charts
TensorFlow / PyTorch	Artificial intelligence and machine learning

WORKED EXAMPLE

Installing and importing Pandas (you will use this in Class XII):

```
pip install pandas
import pandas as pd
```

COMMON MISTAKE

`pip` is a **terminal** command, not a Python statement. Do not type `pip install pandas` inside a `.py` file. In VS Code, open the Terminal panel.

اردو میں سمجھیں

تھرڈ پارٹی لائبریریاں پائتھن کے ساتھ نہیں آتیں، انہیں `pip install` سے انسٹال کیا جاتا ہے۔ نمپائی، پینڈاز، میٹپلاٹ لیب اور ٹینسر فلو اسی قسم کی ہیں۔

CHECK YOURSELF

1. Which library would you choose to draw a bar chart of class marks?
2. Is `math` a third-party library?

Answers: 1. Matplotlib. 2. No — `math` is built-in.

3.7 Debugging Code in Python

LEARN IT

Debugging is the process of identifying, tracing and fixing bugs so a program works correctly. Every program you write will have bugs. The skill is not "never make a mistake"; it is "find the mistake quickly".

Python errors fall into three families:

Type	When it appears	What you notice	Example
Syntax error	Before the program even starts	Python refuses to run	<code>print("Hello" — missing)</code>
Runtime error (exception)	While the program is running	It crashes with a traceback	<code>x = 10 / 0 → ZeroDivisionError</code>
Logical error	Program finishes "successfully"	The output is wrong	<code>area = length + width</code> instead of <code>*</code>

WORKED EXAMPLE

Reading a traceback. When Python crashes it prints a roadmap: the file, the line number, and the error type.

```
x = int("abc")
```

OUTPUT

```
Traceback (most recent call last):
  File "debug_program.py", line 1, in <module>
    x = int("abc")
ValueError: invalid literal for int() with base 10: 'abc'
```

Diagnosis: on **line 1** we tried to convert the letters "abc" into an integer, which raised a `ValueError`.

EXAM TIP

Examiners love "identify the error type" questions. Ask: (1) Does it refuse to start? → syntax. (2) Does it crash mid-run? → runtime. (3) Does it run but give a wrong answer? → logical. Write the name of the error **and** how to fix it.

REMEMBER IT

Fix syntax errors first (the program cannot start). Then run it. Then check the output against a hand-worked example to catch logical errors.

COMMON MISTAKE

A logical error is the most dangerous because Python reports nothing. Always test with values whose answer you already know (for a rectangle 3 by 4, area must be 12, not 7).

اردو میں سمجھیں

ڈیبگنگ یعنی غلطی ڈھونڈ کر ٹھیک کرنا۔ نحوی غلطی پر پروگرام چلتا ہی نہیں، رن ٹائم پر چلتے چلتے رک جاتا ہے، منطقی غلطی پر چل تو جاتا ہے مگر جواب غلط دیتا ہے — یہ سب سے خطرناک ہے۔

CHECK YOURSELF

1. `print("Hi" — which error type?`
2. A program that should add two numbers multiplies them instead. Which error type?
3. `int("12.5")` raises which named exception?

Answers: 1. Syntax error. 2. Logical error. 3. `ValueError` (runtime).

Chapter 3 Review

Chapter summary

- A program is a set of instructions; programming languages are low-level (close to hardware) or high-level (close to English). Python is a high-level, interpreted, readable language.
- Python is widely used in web development, data science, AI and automation — skills that grow a career in Pakistan and abroad.
- An IDE (such as VS Code plus the Python extension) combines an editor, a terminal, a debugger and syntax highlighting.
- Variables store values; Python types are dynamic. Core types: `int`, `float`, `str`, `bool`, `list`, `dict`.
- `print()` displays output; `input()` always returns a **string** — convert with `int()` or `float()` before arithmetic.
- Operators: arithmetic (`+` `-` `*` `/` `//` `%` `**`), assignment (`=` `+=` `...`), relational, logical (`and` `or` `not`), membership (`in`), bitwise (`&` `|` `^` `~` `<<` `>>`).
- Control structures: sequence (top to bottom), selection (`if` / `if-else` / `if-elif-else` / nested), repetition (`for` / `while`).
- `range(stop)` goes 0 ... stop-1; `range(start, stop, step)` jumps by step. `break` leaves a loop; `continue` skips one iteration.
- Use `for` when the count is known; use `while` when you wait for a condition (and always update that condition).
- Libraries extend Python. Built-in: `math`, `random`, `datetime`. Third-party (need `pip`): NumPy, Pandas, Matplotlib, TensorFlow.
- Errors: syntax (grammar, will not run), runtime (crash, traceback), logical (runs, wrong answer).

Key terms

Term	Meaning	اردو
Program	A set of instructions a computer can follow.	پروگرام
High-level language	A language close to English (Python, Java, C++).	اعلیٰ سطحی زبان
Interpreter	Runs a program line by line without a separate compile step.	انٹریپرٹر
IDE	Software that combines editor, terminal and debugger.	انٹیگریٹڈ ڈویلپمنٹ انوائرنمنٹ
Variable	A named box that holds a value.	متغیر
Data type	The kind of value: int, float, str, bool, list, dict.	ڈیٹا کی قسم
Operator	A symbol that performs an operation on operands.	عامل
Operand	A value an operator works on.	معمول
Sequence	Instructions executed from top to bottom.	تسلسل
Selection	Choosing a path with <code>if / else</code> .	انتخاب
Repetition / loop	Repeating a block with <code>for</code> or <code>while</code> .	تکرار / لوپ
Nested	A structure placed inside another of the same kind.	تو در تو
Library / module	A collection of ready-made functions.	لائبریری / ماڈیول
pip	The tool that installs third-party Python libraries.	پپ
Syntax error	A grammar mistake; the program will not start.	نحوی غلطی
Runtime error	A crash while the program is running (exception).	رن ٹائم غلطی
Logical error	The program runs but the result is wrong.	منطقی غلطی
Traceback	Python's report of where a crash happened.	ٹریس بیک
Flag	A Boolean variable that controls a loop or decision.	فلیگ
Integer division	<code>//</code> division that drops the fraction.	صحیح تقسیم

Practice: multiple choice

- Python is best described as a:
 - low-level compiled language
 - high-level interpreted language
 - machine language
 - hardware description language
- `input("Age: ")` always returns a:
 - int
 - float
 - str
 - bool
- The value of `17 // 5` is:
 - 3.4
 - 2
 - 3
 - 3.0
- The value of `17 % 5` is:
 - 2
 - 3
 - 3.4
 - 0
- `2 ** 3` equals:
 - 6
 - 8
 - 9
 - 5
- Which operator is bitwise AND?
 - `and`
 - `&&`
 - `&`
 - `^`
- `list(range(2, 5))` is:
 - `[2, 3, 4, 5]`
 - `[2, 3, 4]`
 - `[2, 5]`
 - `[0, 1, 2, 3, 4]`
- A `while` loop is preferred when:
 - you know the exact number of repeats
 - you must wait for a condition
 - you never need a counter
 - the body must run exactly once
- `break` inside a loop:
 - skips one iteration
 - restarts the loop
 - leaves the loop immediately
 - pauses for input
- `from math import sqrt` lets you write:
 - `math.sqrt(9)` only
 - `sqrt(9)`
 - `import.sqrt(9)`
 - `m.sqrt(9)`
- Which library is built-in?
 - Pandas
 - NumPy
 - random
 - Matplotlib
- `print("Hi")` is a:
 - syntax error
 - runtime error
 - logical error
 - warning only
- `10 / 0` is a:
 - syntax error
 - runtime error
 - logical error
 - style error
- `area = length + width` when you meant to multiply is a:
 - syntax error
 - runtime error
 - logical error
 - import error
- `math.ceil(4.3)` is:
 - 4
 - 4.0
 - 5
 - 4.3

Practice: short questions

1. Differentiate between a compiler and an interpreter.
2. Why must the result of `input()` often be converted with `int()`?
3. State the difference between `/` and `//` with one example each.
4. Write the truth table of the logical operator `and` for two Boolean values.
5. Explain `if-elif-else`. Why is the order of conditions important?
6. Differentiate between a `for` loop and a `while` loop.
7. What do `break` and `continue` do?
8. Name two built-in and two third-party Python libraries and their uses.
9. Differentiate between syntax, runtime and logical errors with one example each.
10. What three pieces of information does a Python traceback give you?

Practice: long / problem questions

1. Write a complete Python program that reads a student's marks (0–100) and prints the grade: 80+ A-1, 70–79 A, 60–69 B, otherwise Fail. Include input conversion and a sample run.
2. Write a program that prints all even numbers from 0 to 10 using `range`, then rewrite it with a `while` loop. Dry-run both.
3. Trace this nested loop and write every line of output:

```
for o in range(1, 3):
    for i in range(1, 4):
        print(o, i)
```
4. A shop gives 20% discount if the bill is 5000 or more. Write a program that keeps reading item prices until the user types 0, prints the total, the discount and the amount to pay. Show a sample run with prices 3000, 2500, 0.
5. Classify each bug and write the corrected line:
 - (a) `print("Total is", total`
 - (b) `avg = total / 0` when there are no items
 - (c) `discount = total + 0.20` when 20% off was intended.
6. Explain how you would use `math`, `random` and `datetime` in a small "lucky student of the day" program for a class of 40. Write the code.

Answer key

Multiple choice

1. b
2. c
3. c
4. a
5. b
6. c
7. b
8. b
9. c
10. b
11. c
12. a
13. b
14. c
15. c

Short questions — model answers

1. A compiler translates the whole program into machine code before it runs (C, C++). An interpreter translates and runs line by line (Python). Compiled programs often run faster; interpreted ones are easier to test.
2. `input()` always returns a string. Adding strings concatenates them (`"2"+"3"` is `"23"`), so numeric work needs `int()` or `float()`.
3. `/` is true division: `17/5` is `3.4`. `//` is integer (floor) division: `17//5` is `3`.
4. $F \text{ and } F \rightarrow F$; $F \text{ and } T \rightarrow F$; $T \text{ and } F \rightarrow F$; $T \text{ and } T \rightarrow T$. Output is True only when both operands are True.
5. Python tests conditions from the top. The first True block runs and the rest are skipped. If you put `marks >= 60` above `marks >= 80`, an 85 would match the weaker condition first and get the wrong grade.
6. `for`: known count, counter updates automatically, good with `range`. `while`: unknown count, you must update the condition yourself or you get an infinite loop.
7. `break` leaves the loop immediately. `continue` skips the rest of this iteration and starts the next one.
8. Built-in: `math` (roots, ceil/floor), `random` (random integers). Third-party: Pandas (tables), Matplotlib (charts). (NumPy and TensorFlow also accepted.)
9. Syntax: missing bracket, will not run. Runtime: `10/0`, crashes. Logical: `area = l + w`, runs but answer is wrong.
10. The error type (e.g. `ValueError`), the file name, and the line number that caused the crash.

Long questions — model answers

```
1. marks = int(input("Enter marks: "))
if marks ≥ 80:
    print("A-1 Grade")
elif marks ≥ 70:
    print("A Grade")
elif marks ≥ 60:
    print("B Grade")
else:
    print("Fail or below B")
```

Sample: input 85 → A-1 Grade; input 72 → A Grade; input 40 → Fail or below B.

```
2. for i in range(0, 11, 2):
    print(i)
n = 0
while n ≤ 10:
    print(n)
    n += 2
```

Both print 0, 2, 4, 6, 8, 10.

3. Output (six lines): 1 1 / 1 2 / 1 3 / 2 1 / 2 2 / 2 3. The inner loop runs fully for each value of the outer loop.

```
4. total = 0.0
while True:
    p = float(input("Price (0 to stop): "))
    if p == 0:
        break
    if p > 0:
        total += p
    discount = total * 0.20 if total ≥ 5000 else 0
    print("Total", total)
    print("Discount", discount)
    print("Pay", total - discount)
```

Sample: 3000 + 2500 = 5500; discount 1100; pay 4400.

5. (a) Syntax — add `)`: `print("Total is", total)`. (b) Runtime `ZeroDivisionError` — only divide when the count of items is not 0. (c) Logical — 20% of total is `total * 0.20`, not `total + 0.20`.

```
6. import random, datetime, math
n = random.randint(1, 40)
today = datetime.datetime.now().strftime("%d-%m-%Y")
print("Lucky roll", n, "on", today)
print("Root of roll", math.sqrt(n))
```

`random` picks the student number, `datetime` stamps the date, `math` shows a sample extra calculation.

Self-assessment: I can...

- | | |
|--|--|
| ☐ classify a language as high-level or low-level and explain why Python is popular. | ☐ declare variables and identify int, float, str, bool, list and dict values. |
| ☐ read input, convert types, and print formatted output. | ☐ evaluate arithmetic, relational, logical and bitwise expressions by hand. |
| ☐ write if, if-else, if-elif-else and nested selections. | ☐ write for and while loops, including range, break and continue. |
| ☐ import built-in libraries (math, random, datetime) in more than one style. | ☐ name pip and at least two third-party libraries. |
| ☐ classify a bug as syntax, runtime or logical and read a traceback. | ☐ combine sequence, selection and repetition in a small complete program. |

